Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION

Name	**:**	**Durgesh Vijay Bhangale**
Week	**:**	**Week 9**
Batch	**:**	**B1_Python**
Trainer	**:**	**Kunal**

MINI PROJECT

1. Smart Expense Tracker (CLI Based)

```
import json
import os
from datetime import datetime

FILE = "expenses.json"
def load_data():
    if os.path.exists(FILE):
        with open(FILE, "r") as f:
            return json.load(f)
    return []
def save_data(data):
    with open(FILE, "w") as f:
        json.dump(data, f, indent=4)
def add_expense(data):
    try:
        amount = float(input("Enter amount: "))
        category = input("Enter category (Food, Travel, etc.): ")
        date = input("Enter date (YYYY-MM-DD): ")
        datetime.strptime(date, "%Y-%m-%d")
        expense = {"amount": amount, "category": category, "date": date}
        data.append(expense)
        save_data(data)
        print("✅ Expense added successfully!")
    except ValueError:
        print("❌ Invalid input. Please try again.")
def view_expenses(data):
    if not data:
        print("No expenses found.")
        return
    print("\n--- All Expenses ---")
    for e in data:
        print(f"{e['date']} | {e['category']} | ₹{e['amount']}")

def filter_category(data):
    cat = input("Enter category to filter: ")
    filtered = [e for e in data if e['category'].lower() == cat.lower()]
    if not filtered:
```

```
        print("No expenses found in this category.")
    else:
        print(f"\n--- Expenses in {cat} ---")
        for e in filtered:
            print(f"{e['date']} | ₹{e['amount']}")
def monthly_summary(data):
    month = input("Enter month (YYYY-MM): ")
    total = sum(e['amount'] for e in data if e['date'].startswith(month))
    print(f"Total spending in {month}: ₹{total}")

def menu():
    data = load_data()
    while True:
        print("\n📊 Smart Expense Tracker")
        print("1. Add Expense")
        print("2. View All Expenses")
        print("3. Filter by Category")
        print("4. Monthly Summary")
        print("5. Exit")
        choice = input("Choose option: ")

        if choice == "1": add_expense(data)
        elif choice == "2": view_expenses(data)
        elif choice == "3": filter_category(data)
        elif choice == "4": monthly_summary(data)
        elif choice == "5":
            print("Goodbye! 👋")
            break
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    menu()
```

2. Student Management System (OOP Based)

```
import json, os

FILE = "students.json"

class Student:
    def __init__(self, name, roll, marks):
        self.name, self.roll, self.marks = name, roll, marks
    def to_dict(self): return {"name": self.name, "roll": self.roll, "marks": self.marks}

def load(): return json.load(open(FILE)) if os.path.exists(FILE) else []
def save(data): json.dump(data, open(FILE,"w"), indent=4)

def add(data):
    name = input("Name: "); roll = input("Roll: "); marks = int(input("Marks: "))
    data.append(Student(name, roll, marks).to_dict()); save(data)

def view(data): [print(s) for s in data]
def search(data):
    roll = input("Enter roll: ")
    print(next((s for s in data if s["roll"]==roll), "Not found"))
def update(data):
    roll = input("Roll to update: ")
    for s in data:
        if s["roll"]==roll: s["marks"]=int(input("New marks: ")); save(data); return
    print("Not found")
def delete(data):
    roll = input("Roll to delete: ")
    data[:] = [s for s in data if s["roll"]!=roll]; save(data)

def menu():
    data = load()
    while True:
        print("\n1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit")
        ch = input("Choice: ")
        if ch=="1": add(data)
        elif ch=="2": view(data)
        elif ch=="3": search(data)
        elif ch=="4": update(data)
```

```
        elif ch=="5": delete(data)
        elif ch=="6": break

menu()
```

3. Library Management System

```
import json, os
FILE="library.json"

def load(): return json.load(open(FILE)) if os.path.exists(FILE) else []
def save(data): json.dump(data, open(FILE,"w"), indent=4)

def add(data):
    name=input("Book name: "); author=input("Author: ")
    data.append({"name":name,"author":author,"issued":False}); save(data)

def issue(data):
    name=input("Book to issue: "); user=input("User name: ")
    for b in data:
        if b["name"]==name and not b["issued"]:
            b["issued"]=True; b["user"]=user; save(data); return
    print("Not available")

def ret(data):
    name=input("Book to return: ")
    for b in data:
        if b["name"]==name and b["issued"]:
            b["issued"]=False; b.pop("user",None); save(data); return
    print("Not found")

def view(data): [print(b) for b in data if not b["issued"]]

def menu():
    data=load()
    while True:
        print("\n1.Add 2.Issue 3.Return 4.View Available 5.Exit")
        ch=input("Choice: ")
        if ch=="1": add(data)
        elif ch=="2": issue(data)
        elif ch=="3": ret(data)
        elif ch=="4": view(data)
        elif ch=="5": break

menu()
```

4. Log Analyzer Tool

```
import re
```

```
with open("system.log") as f: lines=f.readlines()

ips=re.findall(r"\b\d{1,3}(?:\.\d{1,3}){3}\b",""".join(lines))
dates=re.findall(r"\d{4}-\d{2}-\d{2}",""".join(lines))
errors=[l for l in lines if "ERROR" in l]
ip_count={ip:ips.count(ip) for ip in set(ips)}

summary=f"""
Total Errors: {len(errors)}
IPs: {set(ips)}
Dates: {set(dates)}
Error Lines:
{''.join(errors)}
IP Frequency: {ip_count}
"""

print(summary)
open("summary.txt","w").write(summary)
```

5. Task Automation Tool

```
import os

def list_files(path): [print(f) for f in os.listdir(path)]
def rename_files(path):
    for i,f in enumerate(os.listdir(path),1):
        ext=os.path.splitext(f)[1]
        new=f"new_{i}{ext}"
        os.rename(os.path.join(path,f),os.path.join(path,new))
        print(f"{f} → {new}")
def filter_files(path):
    ext=input("Extension (e.g. .txt): ")
    [print(f) for f in os.listdir(path) if f.endswith(ext)]

def menu():
    path=input("Enter folder path: ")
    while True:
        print("\n1.List 2.Rename 3.Filter 4.Exit")
        ch=input("Choice: ")
        if ch=="1": list_files(path)
        elif ch=="2": rename_files(path)
        elif ch=="3": filter_files(path)
        elif ch=="4": break

menu()
```